

Technology Plus

Unit 2: Programming

4.3.2 – Arrays

Lesson Overview:

In this lesson, students will explore how arrays organize data efficiently, practice creating and accessing elements, and understand how arrays differ from other containers like vectors.

Students will:

- Define what an array is and explain its purpose in programming.
- Create, access, and modify array elements.
- Use loops to iterate through arrays.
- Compare arrays to vectors and multidimensional structures.

Guiding Question: How do arrays help us organize and work with data in a program?

Suggested Grade Levels: 8-12

Technology Needed: Yes

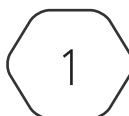
Approximate Time Required: 60-80 minutes

Standards:

CompTIA Tech+ FC0-U71 Objective

- 4.3 – Explain the purpose and use of programming concepts.
 - Identifiers
 - Variables
 - Constants
 - **Arrays**
 - Functions
 - Objects
 - Properties
 - Attributes
 - Methods

This content is based upon work supported by the US Department of Homeland Security's Cybersecurity & Infrastructure Security Agency under the Cybersecurity Education Training and Assistance Program (CETAP).



Lesson Background

Background Information:

Arrays are one of the most fundamental ways programmers store groups of values. Unlike single variables that hold one piece of data, arrays let us organize related values together in order. Because they're stored in contiguous memory, arrays are fast to access but usually have a fixed size. Each value (element) is labeled with an index, typically starting at zero.

Many languages also support vectors or lists, which are similar to arrays but can change size during a program. Arrays are best when the size is known ahead of time; vectors are better when data changes dynamically.

Materials Needed:

Materials per Class	• Lesson Guide • Lesson Slides 4.3.2 – Arrays • Lab Slides 4.3.2 – Array Arcade
Materials per Student	• Student Note 4.3.2 – Arrays • Student Handout 4.3.2 – Arrays • Assessment 4.3.2 – Arrays (optional)

Cyber Terms:

Term	Definition
array	a collection of elements, all of the same type, stored together under one identifier
element	a single value stored inside an array
index	the position number of an element inside an array (starts at 0 in most languages)
multidimensional array	an array containing other arrays, like a grid or table
vector	a container like an array that can grow or shrink in size while a program runs

4.3.2 – Arrays

Guiding Question: How do arrays help us organize and work with data in a program?

Lesson Launch (5-10 minutes)

- Display a picture of a shoe rack (or similar container).
- **Ask:** “How is this organized? What’s the advantage of storing shoes like this instead of scattered around?”
- Connect this to arrays: a rack is like an array, slots are indexes, and shoes are elements.

Exploring Arrays (20-25 minutes)

- Distribute the Student Handout 4.3.2 – Arrays and have students fill in the answers during the lesson.

What is an Array?

An **array** is a collection of elements stored together under one identifier.

- An **element** is a single value inside the array.
- All elements are usually of the same data type (all integers, all strings, etc.).
- Arrays are stored in **contiguous memory locations** – each element sits right next to the others in the computer’s memory.
 - This makes arrays very fast to access in order.
 - But it also means arrays usually have a fixed size once created.
- Each element has an **index** (its position number) used to access it.
- Most languages use **zero-based indexing** → the first element is at index 0.

Analogy: Think of an array as a **shoe rack**. The rack is the array, each slot is an index, and each pair of shoes is an element. The rack is one size when built (fixed size), and each slot is in order right next to the others (contiguous).

Why Use Arrays?

- **Organize Data** – Store related values under one variable instead of many.
- **Efficient Access** – Retrieve or update values quickly using an index.
- **Iteration** – Loop through elements for tasks like searching, sorting, or printing.
- **Structured Storage** – Great for things like a class roster, game scores, or inventory lists.

Working With Arrays

Creating and Accessing Elements

```
scores = [85, 92, 78, 90]    # Array of integers
print(scores[0])             # Output: 85
```

Adding and Removing Elements

```
scores = [85, 92, 78, 90]    # Array of integers
scores.append(99)             # Adds a score of 99
scores.remove(78)             # Removes the score of 78
print(scores)                 # Prints [85, 92, 90, 99]
```

Modifying Elements

```
scores[1] = 95                # Updates the second element
```

Iterating Through Arrays

```
for score in scores:
    print(score)               # Prints each score in the array

for i in range(len(scores)):
    print(scores[i])           # Prints each score in the array based on its index
```

This “loop by index” pattern is handy when you need to work with two arrays in parallel - for example, a players list and a points list where index *i* connects the name and score.

Multidimensional Arrays

Arrays can contain other arrays:

- **2D Array (matrix)** – like a table with rows and columns.
- **3D Array** – like a cube of values.

Example (2D Array in Python):

```
matrix = [
    [1, 3, 5],
    [2, 4, 6],
    [7, 8, 9]
]

print(matrix[1][2])           # Output: 6
```

Arrays vs. Vectors

- **Arrays** – Fixed size, homogeneous elements (all the same type).
- **Vectors** – More flexible, can grow or shrink as the program runs.
 - In C++ and other languages, vectors are the preferred dynamic container.

Examples Across Languages

Python

```
numbers = [1, 2, 3, 4, 5]
print(numbers[2])      # Output: 3
```

JavaScript

```
let numbers = [1, 2, 3, 4, 5];
console.log(numbers[2]); // Output: 3
```

Java

```
int[] numbers = {1, 2, 3, 4, 5};
System.out.println(numbers[2]); // Output: 3
```

Lab: 4.3.2 Array Arcade (30-35 minutes)

- Display Lab Slides 4.3.2 – Array Arcade, have students record their answers in the handout.

Task 1 – High Score Tracker

- Create an array **scores** = [95, 87, 73, 100].
- Print the first and last values.
- Students may use scores[0] and scores[3]. Alternatively, students could also use scores[-1] to get to the last value.
- **Takeaway:** Arrays let us store and retrieve multiple values efficiently.

Task 2 – Inventory Fix

- Array: **inventory** = ["sword", "shield", "potion"].
- Replace "shield" with "magic staff".
- Print updated array.
- **Takeaway:** Individual elements can be updated using their index.

Task 3 – Inventory Corrections

- Array: **computer** = ["monitor", "tower", "mouse", "CD-ROM"].

- Add "keyboard" with append().
- Remove "CD-ROM" with remove().
- Print updated array.
- **Takeaway:** Arrays can grow or shrink while a program runs (Python lists behave like vectors).

Task 4 – Attendance Check

- Array: `students = ["Alex", "Bri", "Chris", "Dana"]`.
- Loop through and print Name is present for each.
- **Takeaway:** Iteration makes it easy to process every element in an array.

Task 5 – Build the Boat

- Build a **2D array** that draws the boat shape using "X" for filled cells and "." for blanks.
- Print with:

```
for row in boat:
    print("".join(row))
```
- **Takeaway:** A 2D array makes it easier for humans to visualize data organized in rows and columns. Computers don't need the structure, but programmers do.

Task 6 – Arcade Leaderboard

- Arrays:

```
players = ["Sam", "Jordan", "Riley"]
points = [1200, 800, 950]
```
- Loop by index to print Sam - 1200, etc.
- **Takeaway:** Parallel arrays connect related data by using the same index.

Tips & Troubleshooting

- **IndexError:** Students may try `scores[4]`. Remind them arrays start at 0.
- **Typos in quotes/brackets:** Watch for missing commas, mismatched quotes, or wrong bracket types.
- **Boat spacing:** If the boat doesn't look right, check whether blanks are "." or " ".
- **Leaderboard pairing:** Some students will try looping by value. Remind them they need an index (i) to match up names and scores.

Putting It All Together

- In Task 1, how did you get the first and last score from the array? Was there another way you could have gotten the last score?
By using indexes (`scores[0]` for the first, `scores[-1]` or `scores[3]` for the last).
- In Task 2 and Task 3, what code let you change or update an array?
We used `inventory[1] = "magic staff"` to replace, `append()` to add, and `remove()` to take something out.

- In Task 5, why do you think we used a 2D array to build the boat instead of one long list of 25 values?
Because the 2D array matches the boat's rows and columns, which makes the code easier for humans to read and organize. The computer doesn't need it that way, but programmers benefit from the structure.
- In Task 6 (Leaderboard), how did the loop know which player went with which score?
We looped by index (for i in range(len(players)):) and used the same i to grab from both arrays.

Putting It All Together (5-10 minutes)

- **Ask:** What is an array?
Sample answer: An array is a collection of elements stored together under one identifier, usually all the same data type.
- **Ask:** How do you access an element?
Sample answer: By using its index number (position). Most languages use zero-based indexing, so the first element is at index 0. Example: scores[0].
- **Ask:** Why are arrays useful compared to individual variables?
Sample answer: They let us organize related values in one place, instead of creating lots of separate variables. They make it easier to loop through data, update values, and keep code more organized and efficient.
- **Ask:** When might you use a vector instead of an array?
Sample answer: When the size of the collection isn't fixed. Vectors (or lists in Python) can grow or shrink as the program runs, which is useful if you don't know how many elements you'll need ahead of time.
- Arrays are the programmer's way of putting order into chaos. Instead of juggling a dozen loose variables, we line them up, give each a number, and suddenly our data is organized, efficient, and ready for action. And when the problem grows beyond rows into grids or changes size as we go, we have tools like multidimensional arrays and vectors to keep things just as neat. Arrays aren't just a coding trick - they're a mindset for organizing information.